

Feed-Forward, GELU & Parameters.

After attention mixes tokens, the feed-forward network refines each token on its own: project up to $4d$, apply a smooth nonlinearity (GELU), project back down. It holds two-thirds of a block's weights. Below we derive GELU exactly via the Gaussian CDF, tabulate it against ReLU, and count every parameter at $d = 768$ — nothing rounded away.

THE EQUATION

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2, \quad \text{GELU}(x) = x \Phi(x)$$

Worked example. At $d = 768$, $d_{\text{ff}} = 3072$: FFN weights $= 2 \cdot 768 \cdot 3072 = 4,718,592$; MHA $= 4 \cdot 768^2 = 2,359,296$; one block $\approx 7.08\text{M}$ parameters.

Topic 3 of 5 in this module · companion to the course page · v1.0

Contents

1	The equation	2
2	GELU, derived exactly	3
3	Parameter count	3
4	Consequences that matter	4
5	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The equation

The position-wise feed-forward network applies the *same* two-layer MLP to every token independently:

$$\text{FFN}(x) = W_2 \text{ act}(W_1 x + b_1) + b_2 \quad (1)$$

with an expand-then-contract shape: $d \rightarrow d_{\text{ff}} \rightarrow d$, conventionally $d_{\text{ff}} = 4d$.

Symbol	Shape	Meaning
x	$\mathbb{R}^d$	one token's vector
W_1	$\mathbb{R}^{d_{\text{ff}} \times d}$	up-projection (expand to d_{ff})
b_1	$\mathbb{R}^{d_{\text{ff}}}$	up bias
W_2	$\mathbb{R}^{d \times d_{\text{ff}}}$	down-projection (contract to d)
b_2	$\mathbb{R}^d$	down bias
act	—	nonlinearity, here GELU

Attention decides *what to read* from other tokens; the FFN decides *what to think* about each token in isolation. The widening to $4d$ gives the nonlinearity room to carve many feature detectors; the projection back to d summarizes them. It is a per-token lookup-and-transform.

2 GELU, derived exactly

GELU (Gaussian Error Linear Unit) gates an input by the probability that a standard normal lands below it:

$$\text{GELU}(x) = x \Phi(x), \quad \Phi(x) = \frac{1}{2} \left(1 + \operatorname{erf} \frac{x}{\sqrt{2}} \right) \quad (2)$$

Φ is the standard-normal CDF, so the *exact* GELU uses the error function erf . A common closed-form approximation avoids erf :

$$\text{GELU}(x) \approx \frac{1}{2} x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right] \right). \quad (3)$$

By contrast, $\text{ReLU}(x) = \max(0, x)$ is a hard gate (0 or 1). GELU is a *soft* gate: $\Phi(x)$ slides smoothly from 0 to 1, so small negative inputs are damped rather than killed, and the function is differentiable everywhere.

Tabulated, computed exactly via erf:

x	$\Phi(x)$	$\text{GELU}_{\text{exact}}$	$\text{GELU}_{\text{tanh}}$	ReLU
-2	0.022750	-0.045500	-0.045402	0.0
-1	0.158655	-0.158655	-0.158808	0.0
0	0.500000	0.000000	0.000000	0.0
1	0.841345	0.841345	0.841192	1.0
2	0.977250	1.954500	1.954598	2.0

Note $\text{GELU}(-1) = -0.158655$ versus $\text{ReLU}(-1) = 0$: GELU lets a little negative signal through. The exact and tanh forms agree to three decimals. Heatmap of $|\text{GELU}_{\text{exact}}(x)|$ (intensity scales with magnitude, cap 80):

	$x=-2$	$x=-1$	$x=0$	$x=1$	$x=2$
GELU	-0.0455	-0.1587	0.0000	0.8413	1.9545

GELU = $x \Phi(x)$: the input scaled by “how far above the mean of a unit Gaussian it sits.” Far positive $\Rightarrow \Phi \rightarrow 1 \Rightarrow$ near-identity; far negative $\Rightarrow \Phi \rightarrow 0 \Rightarrow$ near-zero; near 0 a smooth, differentiable blend. This smoothness is why GELU replaced ReLU in BERT/GPT-class transformers.

3 Parameter count

Count the learnable weights. The FFN’s two matrices dominate:

$$|W_1| = d_{\text{ff}} \cdot d, \quad |W_2| = d \cdot d_{\text{ff}}, \quad |W_1| + |W_2| = 2 d d_{\text{ff}} = 2d(4d) = 8d^2,$$

plus biases $|b_1| + |b_2| = d_{\text{ff}} + d = 5d$ (negligible). Multi-head attention uses four $d \times d$ projections (W_Q, W_K, W_V, W_O):

$$\text{MHA weights} = 4 d^2.$$

So a block's weight count is dominated by

$$\text{block} \approx \underbrace{8d^2}_{\text{FFN}} + \underbrace{4d^2}_{\text{MHA}} = 12d^2 \text{ (+ norms)} \quad (4)$$

LayerNorm adds only $2 \times 2d = 4d$ parameters per block (two norms, each with $\gamma, \beta \in \mathbb{R}^d$).

Worked at $d = 768$, $d_{\text{ff}} = 3072$:

Component	Count	Working
W_1	2,359,296	3072×768
W_2	2,359,296	768×3072
FFN weights	4,718,592	$2 \cdot 768 \cdot 3072 = 8d^2$
FFN biases	3,840	$3072 + 768$
MHA weights	2,359,296	$4 \cdot 768^2$
$2 \times$ LayerNorm	3,072	$2 \cdot 2 \cdot 768$
Block (weights, $12d^2$)	7,077,888	$= 7.078\text{M}$
Block (incl. biases + norms)	7,084,800	$= 7.085\text{M}$

The FFN holds $4,718,592/7,077,888 = 66.7\%$ of the block's matrix weights — exactly two-thirds, the direct consequence of $8d^2$ vs $4d^2$.

4 Consequences that matter

1. **FFN is the heavyweight.** At $d_{\text{ff}} = 4d$ the FFN carries twice the MHA parameters; scaling d_{ff} is the cheapest knob for capacity.
2. **Quadratic in d .** Every block costs $\approx 12d^2$ params, so doubling width quadruples per-block size.
3. **Per-token, weight-shared.** The same W_1, W_2 apply to all T positions; the FFN adds no parameters as sequences grow.
4. **GELU is differentiable everywhere**, including at 0, unlike ReLU's kink — gradients are smooth.

Two traps. (i) $d_{\text{ff}} = 4d$ is a *convention*, not a law — T5 uses different ratios and gated variants (GeGLU/SwiGLU) use three matrices with $d_{\text{ff}} \approx \frac{8}{3}d$ to keep the parameter count matched. Always check the config. (ii) The exact GELU needs erf; the tanh form (3) is an *approximation* — mixing the two between training and inference produces tiny but real discrepancies (GELU(−1): −0.158655 vs −0.158808).

5 Where this sits in the track

Link	Connection
Module 0.1 / 0.2 T-others	MHA supplies the other $4d^2$; together they form the block.
Topic 2 (this module)	The two LayerNorms counted above.
Module 3.1	SwiGLU replaces GELU; explains the $\frac{8}{3}d$ width adjustment.
Phase 1 scaling	$12d^2$ per block $\times$ depth gives total model size.

A Reproducible (NumPy)

Inputs: $d = 768$, $d_{\text{ff}} = 3072$; GELU via $\Phi(x) = \frac{1}{2}(1 + \text{erf}(x/\sqrt{2}))$. Key outputs: FFN = 4,718,592, MHA = 2,359,296, block weights = 7,077,888.

```
import numpy as np
from math import erf, sqrt, pi, tanh
Phi = lambda x: 0.5*(1+erf(x/sqrt(2)))
gelu = lambda x: x*Phi(x)
gelu_tanh = lambda x: 0.5*x*(1+tanh(sqrt(2/pi)*(x+0.044715*x**3)))
relu = lambda x: max(0.0, x)
for x in (-2,-1,0,1,2):
    print(x, round(Phi(x),6), round(gelu(x),6),
          round(gelu_tanh(x),6), relu(x))
# x=-1 -> Phi 0.158655 GELU -0.158655 tanh -0.158808 ReLU 0.0
# x= 2 -> Phi 0.977250 GELU 1.954500 tanh 1.954598 ReLU 2.0

d, dff = 768, 3072
ffn = 2*d*dff; mha = 4*d*d; ln = 2*2*d
print(ffn, mha, ffn+mha) # 4718592 2359296 7077888 (== 12*d*d)
```